

Nhập môn Công nghệ phần mềm

Chương 4
Lập trình

Lập trình

- Tổng quan
- Phương pháp lập trình
- Ngôn ngữ lập trình
- Phong cách lập trình
- Kỹ thuật lập trình
- Lập trình hướng hiệu quả
- Tổng kết

Phương pháp lập trình

- Lập trình tuần tự (tuyến tính)
- Lập trình có cấu trúc (thủ tục)
- Lập trình hướng chức năng
- Lập trình hướng đối tượng OOP
- Lập trình Logic
- Lập trình hàm

Lập trình – Phương pháp LT

Lập trình tuần tự (tuyến tính)

- Không có/thiếu các lệnh có cấu trúc (for, while, do while) *ASS*
- Lạm dụng các lệnh GOTO
- Thiếu khả năng khai báo biến cục bộ *. - ↓
o*
- Độ ghép nối cao *END*
- Do đó chương trình khó hiểu, khó sửa, dễ sinh lỗi
- Vd: Fortran, Assembler, Basic,... (*từ đó*)

Lập trình – Phương pháp LT

- VD về chương trình Fortran

DIMENSION A(10)

....

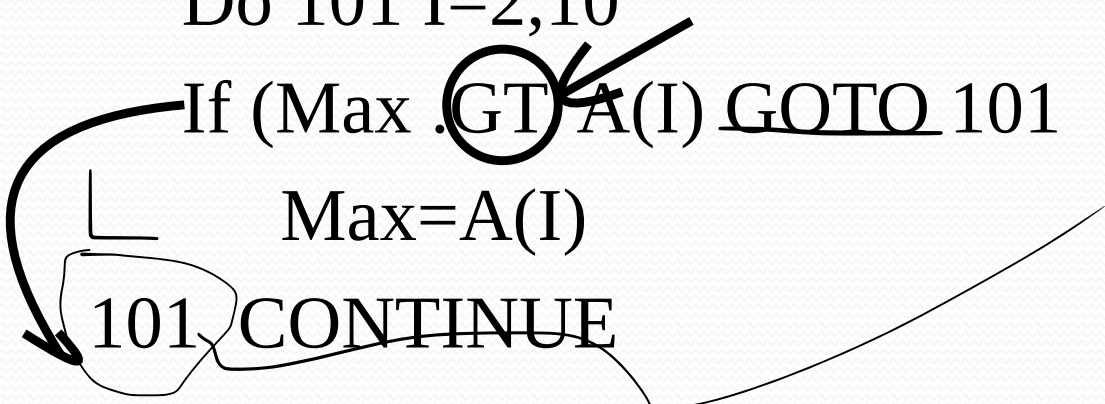
Max=A(1)

Do 101 I=2,10

If (Max .GT. A(I)) GOTO 101

└ Max=A(I)

101 CONTINUE



Lập trình – Phương pháp LT

Lập trình cấu trúc

- Sử dụng các lệnh có cấu trúc (for, while, do while)
- Hạn chế/cấm dùng GOTO (C, VB, Net, VB6)
- Sử dụng chương trình con, biến cục bộ. Do đó dễ hiểu hơn, an toàn hơn lập trình tuyến tính
- VD: Pascal, C, ...

Lập trình – Phương pháp LT

Lập trình hướng chức năng

- Dựa trên nguyên tắc ghép nối dữ liệu
 - Trao đổi dữ liệu bằng tham số và giá trị trả lại
 - Loại bỏ hoàn toàn dữ liệu dùng chung
- Do đó loại bỏ các hiệu ứng phụ khi sửa đổi các modul chương trình
- Nâng cao tính tái sử dụng
- VD: Modula, ..

$f(a, b) \rightarrow \text{int}$

$f_1() \rightarrow$

SOA $\rightarrow$ function $(p_1, p_2, \dots)$

Lập trình – Phương pháp LT

Lập trình hướng đối tượng

- Che dấu thông tin (
- Thao tác với dữ liệu qua các giao tiếp (interface) xác định (lồng lỏ) SWD
- Kế thừa

- Cục bộ hơn
- Dễ tái sử dụng hơn
- Thuận tiện cho các ứng dụng lớn

- VD: SmallTalk, C++, Java, DotNet (C#), Perl, Node, Python → Net FW, 4.0, 3.5

Lập trình – Phương pháp LT

Lập trình Logic

- Tách tri thức về bài toán khỏi kỹ thuật lập trình

- Mô tả tri thức

- Các qui tắc

- Các sự thực

- Mục tiêu

- Hệ thống tự chứng minh

- Tìm đường đi đến mục tiêu

- Không có khái niệm về vùng nhớ, biến

- Biến là biến như toán học.

- Ví dụ: Prolog

mail | ab

mat lab

+

Rất hay | 1 bod

9 | Bit

Lập trình – Phương pháp LT

- Ví dụ về prolog

⇒ “Là người thì phải chết, Socrate là người nên phải chết”

PhaiChet(x):-Nguoi(x).

Nguoi(socrate)

Nguoi(ba)

Nguoi(tu)

Goal PhaiChet(socrate)

Yes

Goal PhaiChet(Ai)

Ai=socrate

Ai=ba

Ai=tu

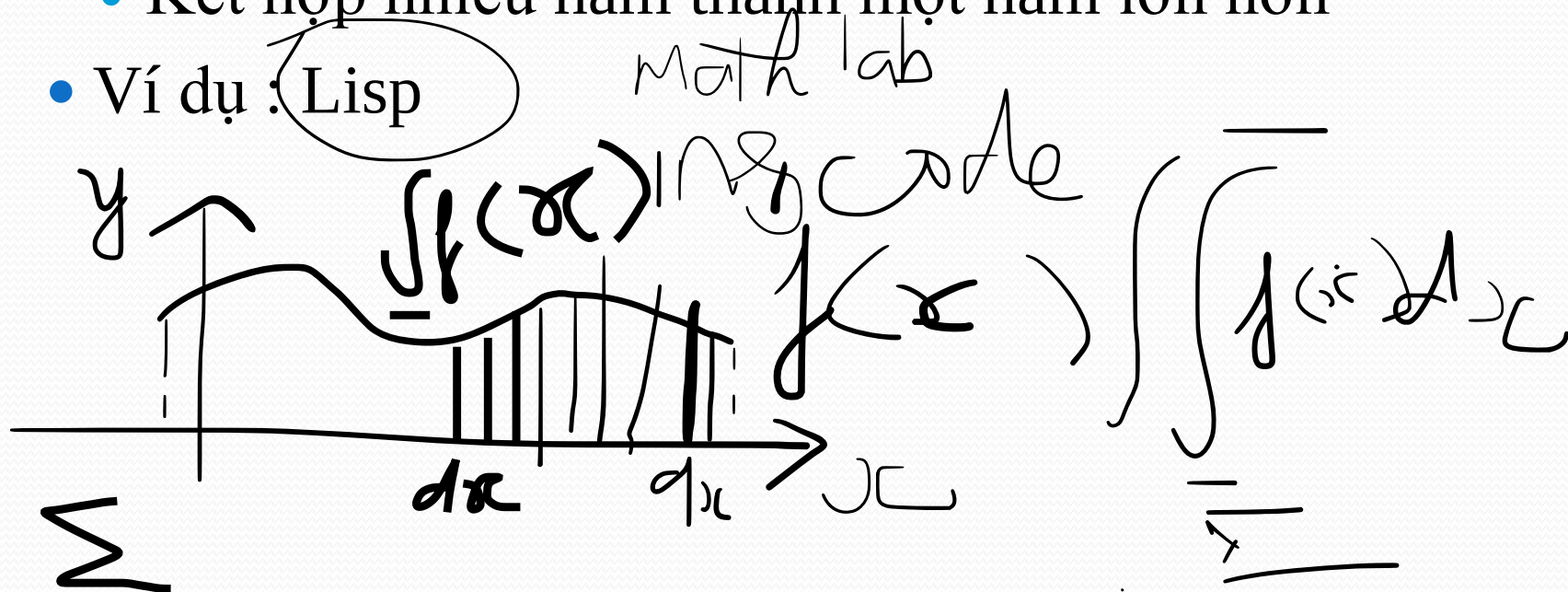
ai socrate

Lập trình – Phương pháp LT

Lập trình Hàm

- Sự trở về với toán học
- Hàm phải là một hàm toán học
 - Mỗi modul chương trình là một hàm
 - Kết hợp nhiều hàm thành một hàm lớn hơn
- Ví dụ : Lisp

$$f(x) = \frac{\sin(x)}{\cos(x)}$$



Lập trình – Phương pháp LT

- Ví dụ về lisp

```
(DEFUN LuyThua (M N)
```

```
  (COND ((EQ N 0) 1)
```

```
    (T(TIMES M (LuyThua M (DIEF N 1))))))
```

$N \rightarrow \uparrow$

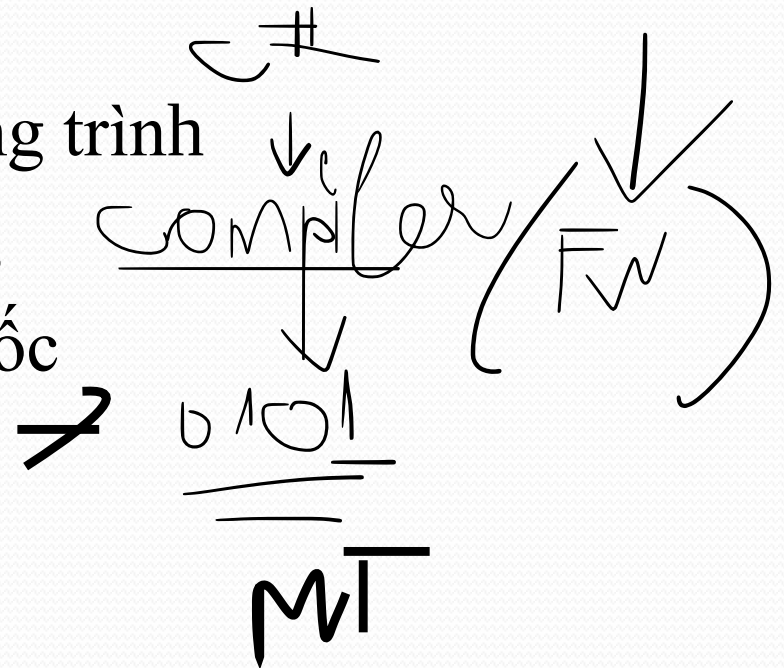
Ngôn ngữ lập trình

Ngôn ngữ lập trình tốt

- Có thể thiết lập được một tập hợp tổng quát những yêu cầu như sau:

- Dễ dịch thiết kế sang chương trình
- Có trình biên dịch hiệu quả,
- Khả chuyển chương trình gốc
- Có sẵn công cụ phát triển
- Dễ bảo trì

$\text{SWD} \rightarrow \text{SWP}$



Lập trình – NNLT tốt

- Dịch thiết kế sang chương trình
 - Về lý thuyết, việc sinh chương trình gốc từ một đặc tả chi tiết nên là trực tiếp. $RS \rightarrow Program$
 - Tính dễ dịch thiết kế sang chương trình sẽ cho phép một ngôn ngữ cài đặt trực tiếp cho các kết cấu có cấu trúc, các cấu trúc dữ liệu phức tạp, các vào/ra đặc biệt (thao tác trên bit) và các đối tượng.

○ 1

- Làm cho việc dịch từ thiết kế sang chương trình gốc dễ hơn nhiều. $SWD \rightarrow SWP$

Lập trình – NNLT tốt

10, CPU 32R

204 Hz

• Có trình biên dịch hiệu quả RAM, SSD, CPU

- Mặc dù những tiến bộ nhanh chóng trong tốc độ xử lý và mật độ bộ nhớ đã làm giảm nhẹ nhu cầu chương trình siêu hiệu quả. Tuy nhiên các chương trình chạy nhanh, gọn (chiếm ít tài nguyên hệ thống) luôn được đánh giá cao. *Build* →
- Đối với những phần mềm mà hiệu năng là yêu cầu quan trọng nhất thì các ngôn ngữ với trình biên dịch tối ưu sẽ được ưu tiên lựa chọn.

C, ASS → ΔT <<

Lập trình – NNLT tốt

- Khả chuyển chương trình gốc
 - Có thể được chuyển từ nền tảng (hệ thống) này sang nền tảng kia với rất ít hoặc không phải sửa đổi gì.

• Ví dụ:

LINUX, WINDOWS, Mac

- Java: write once, run anywhere
- .Net: NetFramework ↔ JVM (Java Virtual Machine)

2014 → Net Core 3.1

Lập trình – NNLT tốt

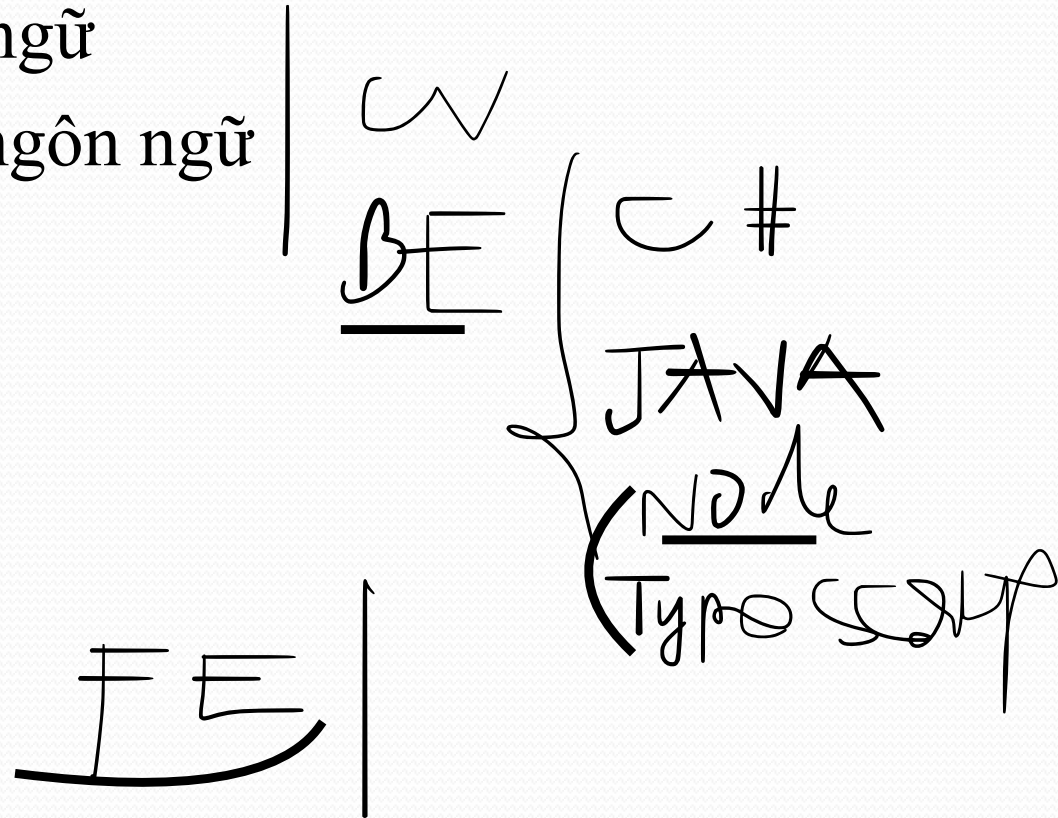
- Có sẵn công cụ phát triển
 - { • Làm ngắn bớt thời gian cần để sinh ra chương trình gốc và có thể cải thiện chất lượng của chương trình.
 - Trong thực tế, khái niệm về môi trường phát triển phần mềm tốt (bao hàm cả các công cụ) đã được thừa nhận như nhân tố đóng góp chính cho công nghệ phần mềm thành công. *Công A Dựng*
- Dễ bảo trì
 - Có tầm quan trọng chủ chốt cho tất cả các nỗ lực phát triển phần mềm. *(→ 20)*

Lập trình – NNLT tốt

- Việc bảo trì được tiến hành khi người ta hiểu được thiết kế chi tiết phần mềm và cuối cùng chương trình gốc vẫn phải được đọc và sửa đổi. ←
- Tính dễ dịch thiết kế sang chương trình là một yếu tố quan trọng để dễ bảo trì chương trình gốc. Bên cạnh đó, các đặc trưng về cấu tạo của ngôn ngữ (như chiều dài của tên gọi, định dạng nhãn, định nghĩa kiểu, cấu trúc dữ liệu) có ảnh hưởng mạnh đến tính dễ bảo trì. →

Ngôn ngữ lập trình

- Đặc trưng của ngôn ngữ
- Miền ứng dụng của ngôn ngữ
- Lựa chọn ngôn ngữ



Lập trình – Ngôn ngữ lập trình

Đặc trưng của ngôn ngữ

- Năng lực (kiểu biến, các cấu trúc)
- Tính khả chuyển
- Mức độ hỗ trợ của các công cụ

Lập trình – Đặc trưng của NN

Năng lực của ngôn ngữ

- Có cấu trúc, câu lệnh phong phú
- Hỗ trợ nhiều kiểu dữ liệu
- Hỗ trợ con trỏ, đệ qui $\subset^{++}$
- Hỗ trợ hướng đối tượng
- Thư viện phong phú
- Do đó nên chọn ngôn ngữ bậc cao

Handwritten notes and symbols:

- A large curly brace grouping the first two bullet points.
- Handwritten symbols: $\{ \# \sqrt{}$ and $P \times G$.
- Handwritten text: "Rip" and "PM".

Lập trình – Đặc trưng của NN

Tính khả chuyển

- Thay đổi phần cứng
- Thay đổi hệ điều hành
- Nhận xét:
 - C khả chuyển mã nguồn giữa các hệ UNIX
 - Java khả chuyển $\subset J \vee M$
 - Các ngôn ngữ script khả chuyển

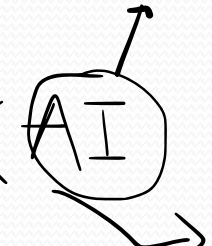
Lập trình – Đặc trưng của NN

Hỗ trợ của công cụ

- Trình biên dịch hiệu quả
 - Biên dịch tốc độ cao
 - Khả năng tối ưu cao
 - Khai thác các tập lệnh, kiến trúc phần cứng mới
- Các công cụ trợ giúp hiệu quả
 - Editor, **debugger** linker, make...
 - IDE (Integrated Development Environment)

Lập trình – Đặc trưng của NN

Miền ứng dụng

- Phần mềm hệ thống:
 - Hiệu quả, vận năng, dễ mở rộng
 - Vd: C
- Hệ thời gian thực: C, C++, Ada, Assembler
- Phần mềm nhúng: C++, Java 
- Phần mềm khoa học kỹ thuật:
 - Tính toán chính xác, thư viện toán học mạnh,
 - Dễ dàng song song hóa
 - Fortran vẫn phổ biến, Matlab, .. *Python* ()

Lập trình – Đặc trưng của NN

- Phần mềm nghiệp vụ:
 - CSDL: Oracle, DB2, SQL Server, MySQL...
 - Ngôn ngữ: FoxPro, COBOL, VB, VC++

- Trí tuệ nhân tạo:

- Lisp, Prolog, OPS5,...

deep

NN

LLM

- Lập trình Web/CGI:

JSP

- Perl, ASP, PHP, Java, Java script, Python..

FW
↓
Laravel

Lập trình – Đặc trưng của NN

Lựa chọn ngôn ngữ

- Dựa vào:
 - Đặc trưng của ngôn ngữ
 - Miền ứng dụng của ngôn ngữ
 - Năng lực, kinh nghiệm của nhóm phát triển
 - Yêu cầu của khách hàng (~~JA~~ ~~VA~~)
- Một số nhận xét
 - Phát triển phần mềm hệ thống (C)
 - Thời gian thực Ada, C, Assembler
 - Phần mềm nhúng Java

Lập trình – Đặc trưng của NN

- Một số nhận xét (tt)

↑ Toán Python

- Khoa học kỹ thuật : MatLab, Fortran, Algol

- Kinh doanh : Cobol BI

- Trí tuệ nhân tạo : Lisp, Prolog Python

- Phần mềm hướng đối tượng : Smalltalk, C++, Java, Eiffel, Object- PASCAL, Flavos và nhiều ngôn ngữ khác. C#

- Web: PHP, ASP, Java, JavaScript, PERL...

React, Angular2,

Lập trình – Đặc trưng của NN

Ảnh hưởng của NNLT và CNPM

- Thiết kế phần mềm được thiết lập theo cách độc lập với các đặc trưng ngôn ngữ lập trình
- Thiết kế dữ liệu cũng có thể bị ảnh hưởng bởi các đặc trung ngôn ngữ. Oracle, SQL
- Các đặc trưng của ngôn ngữ cũng ảnh hưởng tới kiểm thử phần mềm | SWT

Phong cách lập trình

Phong cách lập trình tốt

- Tuân theo các chuẩn thông dụng
- Chuẩn được chấp nhận rộng rãi hơn dễ hiểu hơn
- Chú giải đầy đủ mỗi khi không tuân theo chuẩn

Phong cách lập trình

- Chú thích bên trong chương trình /
- Phương pháp khai báo dữ liệu
- Cách xây dựng câu lệnh
- Các kĩ thuật vào/ra
- Xử lý lỗi

Lập trình – Chú thích

- Cung cấp cho người phát triển một ý nghĩa truyền thông với các độc giả khác về chương trình gốc /// ρ
- Cung cấp một hướng dẫn rõ rệt để hiểu trong pha cuối cùng của công nghệ phần mềm - bảo trì
- Thường gồm 2 loại:
 - Chú thích mở đầu < '
 - Chú thích chức năng →

Lập trình – Chú thích

- Chú thích đầu chương trình
 1. Một phát biểu về mục đích chỉ rõ chức năng mô đun.
 2. Mô tả giao diện bao gồm:
 - Một mẫu cách gọi
 - Mô tả về dữ liệu
 - Danh sách tất cả các mô đun thuộc cấp
 3. Thảo luận về dữ liệu thích hợp (như các biến quan trọng và những hạn chế, giới hạn về cách dùng chúng) và các thông tin quan trọng khác.
 4. Lịch sử phát triển bao gồm:
 - Tên người thiết kế modul (tác giả).
 - Tên người xét duyệt và ngày tháng.
 - Ngày tháng sửa đổi và mô tả sửa đổi.

Lập trình – Chú thích

```
/**
 * A trading individual evolved by the GP. The Individual contains a buy rule and
 * a sell rule that will have been evolved by the GP.
 *
 * @author Andrew Leppard
 * @see GeneticProgramme
 * @see Mutator
 */
```

Handwritten signature: *Andrew Leppard*

```
public class Individual implements Comparable {
```

```
> // The evolved buy indicator
  private Expression buyRule = null;
```

Handwritten: *// #*

```
  // The evolved sell indicator
  private Expression sellRule = null;
```

```
  // The individuals portfolio after trades
  private Portfolio portfolio = null;
```

```
  // The value of the portfolio
  private Money value = null;
```

Lập trình – Khai báo

- **Phương pháp khai báo dữ liệu**

- Tên gọi có nghĩa cũng làm tăng tính dễ hiểu
- Đặt tên biến, tên hàm có nghĩa, gợi nhớ
- Sử dụng các ký hiệu, từ có nghĩa làm cho dễ đọc. Ví dụ: dùng DateOfBirth hoặc date_of_birth không viết dateofbirth
- Tránh đặt tên quá dài
- Thống nhất cách dùng. Ví dụ I cho vòng lặp, Tmp cho các giá trị tạm thời...
- Các tên biến ngoài việc có nghĩa còn nên mang thông tin về kiểu của chúng
vd: strHoTen, intStt
- Cần phải chú giải về mục đích đối với các biến quan trọng, đặc biệt là các biến toàn cục.
- Các cấu trúc dữ liệu nên được chú giải đầy đủ về cấu trúc và chức năng.

Lập trình – Câu lệnh

- Xây dựng câu lệnh
 - Các câu lệnh phải mô tả cấu trúc
 - Tụt lề, dễ đọc, dễ hiểu
 - Làm đơn giản các lệnh
 - Mỗi lệnh trên một dòng
 - Triển khai các biểu thức phức tạp
 - Tránh các điều kiện phức tạp) { ()
 - Hạn chế truyền tham số là kết quả của hàm, biểu thức (vd : `printf("%s", strcpy(des, src));`)

→ `S = strcpy(P)`

Lập trình – Câu lệnh

- Tránh các cấu trúc phức tạp
 - Các lệnh if lồng nhau
 - Điều kiện phủ định ~~if not~~
- Dùng dấu ngoặc để làm sáng tỏ biểu thức Logic hay số học $(a > b)$ ~~$(c == b)$~~
- Chỉ dùng các tính năng chuẩn của ngôn ngữ

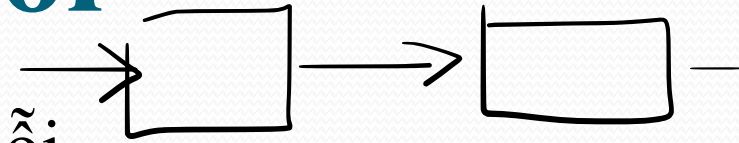
Lập trình – Vào/ra

- **Kỹ thuật vào/ra của các môđun:** nên tuân thủ theo ~~một số hướng dẫn sau:~~
 - Làm hợp lệ mọi dữ liệu vào. (null)
 - Kiểm tra sự tin cậy của các tổ hợp dữ liệu vào quan trọng.
 - Giữ cho định dạng dữ liệu vào đơn giản.
 - Giữ cho định dạng dữ liệu vào thống nhất khi một ngôn ngữ lập trình có các yêu cầu định dạng nghiêm ngặt. (dd/mm/yyyy)

Lập trình – Xử lý lỗi

- Có thể phát hiện lỗi trong khi thực hiện
 - Lỗi chia 0
 - Lỗi input/output, ...
- Xử lý lỗi
 - Nhất quán trong xử lý
 - Phân loại lỗi, thống nhất định dạng thông báo,...
 - Phân biệt output và thông báo lỗi
 - Các hàm thư viện nên tránh việc tự xử lý tự đưa ra thông báo lỗi

Lập trình – Xử lý lỗi



- Phân biệt Output và thông báo lỗi
 - Output là dữ liệu, còn được dùng để làm input cho phần mềm khác $f()$
 - Thông báo (lỗi) là các thông tin nhất thời, trạng thái hệ thống, lỗi và cách khắc phục
 - OS thường cung cấp 3 luồng dữ liệu chuẩn
 - stdin (cin): input chuẩn (bàn phím)
 - stdout (cout): output chuẩn (màn hình)
 - stderr (cerr): luồng thông báo lỗi chuẩn

Lập trình – Xử lý lỗi

- Hàm thư viện trả lại trạng thái lỗi, không tự xử lý
 - Người viết và người sử dụng thư viện là khác nhau
 - Người sử dụng thường muốn có cách xử lý riêng
 - Trả trạng thái bằng giá trị trả lại
 - Trả trạng thái bằng tham số
 - Trả lại bằng ném ngoại lệ (trong các OOL)

Lập trình – Xử lý lỗi

- **Xử lý lỗi**

- Ngoại lệ:

- Là cách thức xử lý lỗi tiến tiến trong các ngôn ngữ hướng đối tượng
 - môđun xử lý ném ra một ngoại lệ (đối tượng chứa thông tin lỗi)
 - môđun điều khiển bắt ngoại lệ (nếu có)
 - Tách phần xử lý lỗi khỏi phần cài đặt thuật toán thông thường, làm cho chương trình dễ đọc hơn
 - Dễ dùng hơn, an toàn hơn

Lập trình – Xử lý lỗi

- Ví dụ: Ngoại lệ (ném ngoại lệ)

```
double MyDivide(double num, double denom)
```

```
{
```

```
if (denom == 0.0) {
```

```
    throw invalid_argument("The denom cannot be  
    0.");
```

```
}
```

```
else {
```

```
    return num / denom;
```

```
} }
```

Lập trình – Xử lý lỗi

- Ví dụ: Ngoại lệ (bất ngoại lệ)

```
try {  
    result = MyDivide(x, y);  
}  
catch (invalid_argument& e) {  
    cerr << e.what() << endl;  
    ... // mã xử lý với ngoại lệ  
};
```

Lập trình tránh lỗi

- Mục tiêu: xây dựng hệ thống tin cậy
 - Tránh lỗi: Phần mềm được phát triển theo cách để nó không chứa lỗi
 - Phòng thủ: Luôn giả định rằng hệ thống có lỗi. Chương trình chứa mã để phát hiện và có thể quay trở về trạng thái trước khi có lỗi.
 - Thử lỗi: Phần mềm được thiết kế sao cho hệ thống vẫn hoạt động mặc dù phần mềm có lỗi (try ... catch).

Lập trình – Lập trình tránh lỗi

- Kỹ thuật lập trình tốt dựa trên các yếu tố:
 - Lập trình có cấu trúc
 - Dùng các lệnh có cấu trúc
 - Môđun hóa
 - Hạn chế dùng các cấu trúc nguy hiểm
 - Đóng gói/che dấu thông tin
 - Hạn chế dữ liệu dùng chung
 - Hạn chế thao tác trực tiếp lên thuộc tính

Lập trình – Lập trình tránh lỗi

- Tránh các cấu trúc nguy hiểm
 - Số thực
 - Các phép toán được làm tròn, kết quả không chính xác tuyệt đối
 - So sánh (bằng nhau) hai số thực là không khả thi
 - Con trỏ
 - Khái niệm mức thấp
 - Có khả năng gây lỗi nghiêm trọng
 - Dễ nhầm
 - `double r;`
 - `int* n = &r;`

Lập trình – Lập trình tránh lỗi

- Cấp phát bộ nhớ động
 - Quên cấp phát
 - Quên giải phóng
 - Chỉ nên dùng với ngôn ngữ hướng đối tượng (C++)
- Độ qui
 - Khó hiểu
 - Dễ nhầm điều kiện dừng

Lập trình – Lập trình phòng thủ

- Defensive programming
 - Dự đoán khả năng xuất hiện lỗi
 - Khắc phục lỗi
 - Lưu trạng thái an toàn
 - Quay lại trạng thái an toàn gần nhất
- Nhiều lệnh có khả năng sinh lỗi
 - Lệnh vào/ra
 - Các phép toán
 - Thao tác với bộ nhớ
 - Truyền tham số sai kiểu

Lập trình – Lập trình phòng thủ

- Lệnh vào ra
 - Dữ liệu không hợp lệ
 - Tràn bộ đệm (kiểu ký tự)
 - Lỗi thao tác file (sai tên, chưa được mở,...)
- Các phép toán
 - Lỗi chia 0
 - Tràn số
 - So sánh số thực (bằng nhau)

Lập trình – Lập trình phòng thủ

- Thao tác bộ nhớ
 - Quên cấp phát, quên giải phóng bộ nhớ
 - Thiếu bộ nhớ
 - Sai địa chỉ, tràn bộ nhớ

```
FILE* fp;  
fp = fopen("data",  
"r");
```



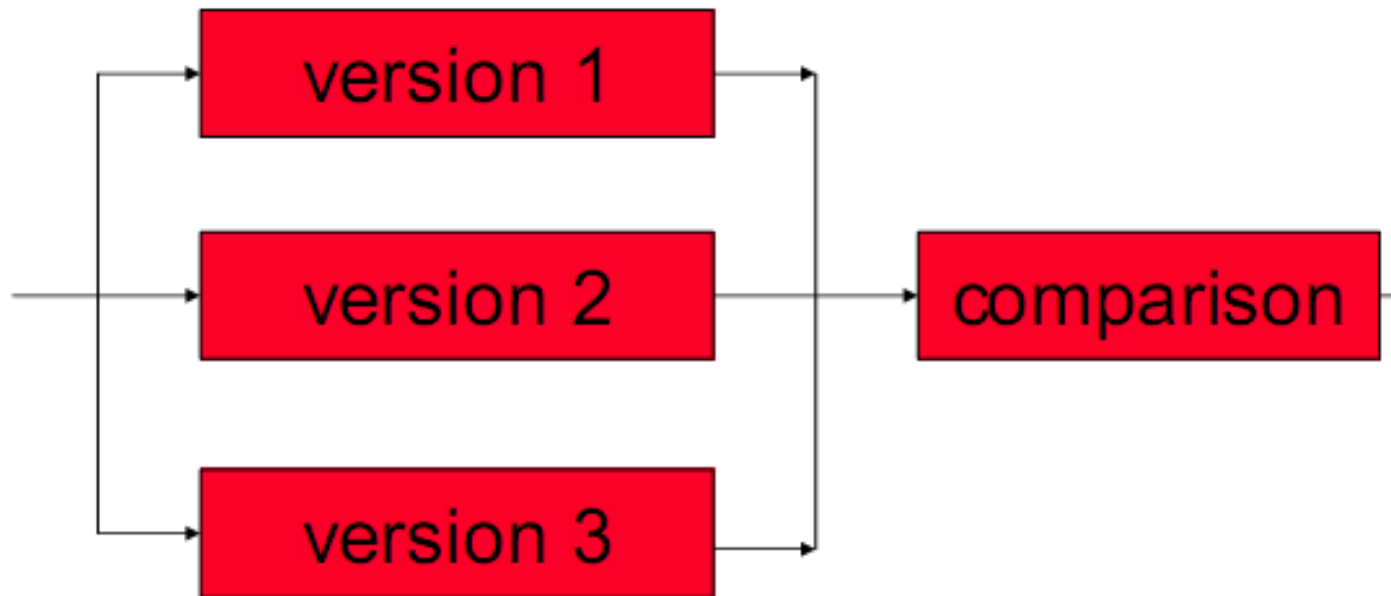
```
FILE* fp;  
if (NULL == (fp = fopen("data",  
"r"))) {  
    fprintf(stderr, "can not  
open file...");  
    ...  
}
```

Lập trình – Thử lỗi

- Thử lỗi
 - Fault tolerance programming
 - Phát hiện lỗi
 - Định ra mức độ thiệt hại
 - Hồi phục sau khi gặp lỗi
 - Chữa lỗi

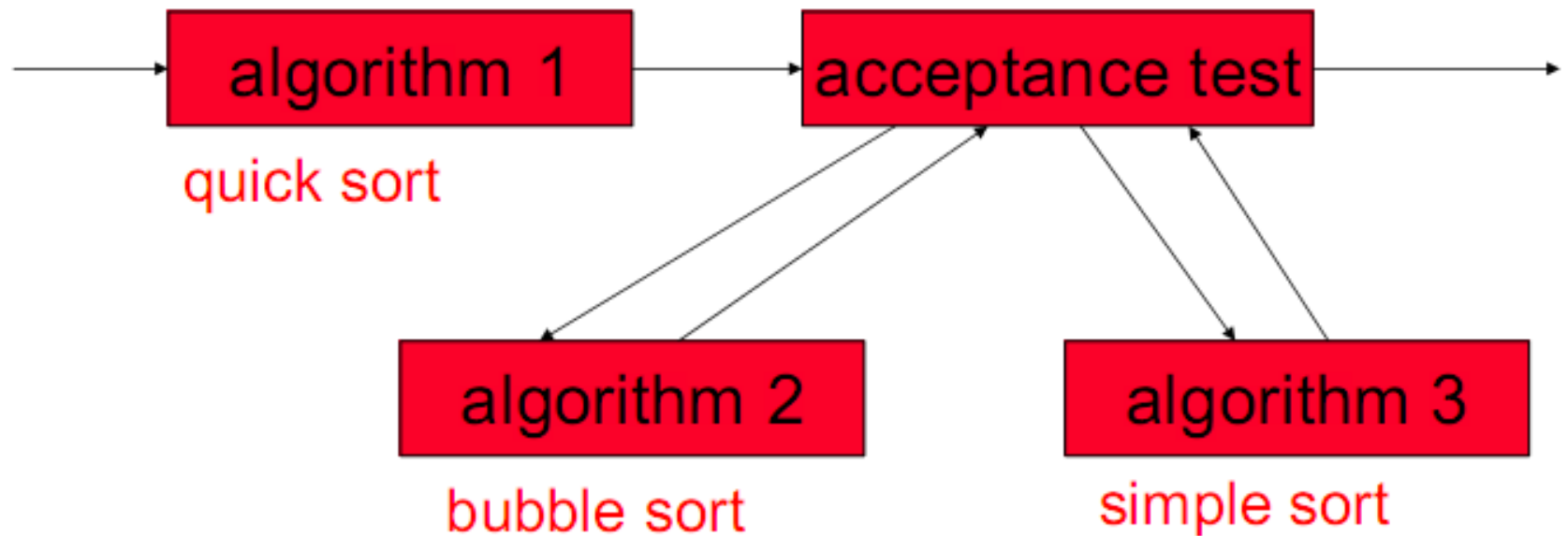
Lập trình – Thử lỗi

- N-Version
 - Sử dụng nhiều nhóm lập trình
 - So sánh kết quả
 - Cùng thuật toán thì có xu hướng mắc cùng lỗi



Lập trình – Thử lỗi

- Recovery blocks
 - Sử dụng các thuật toán khác nhau
 - Kiểm tra tính hợp lệ của kết quả



Lập trình hướng hiệu quả

Lập trình hướng hiệu quả khi thực hiện chương trình

- Tính hiệu quả chương trình
 - Liên quan trực tiếp với tính hiệu quả của thuật toán được xác định trong thiết kế chi tiết.
 - Phong cách lập trình có thể tác động đến tốc độ thực hiện và yêu cầu bộ nhớ.
 - Các hướng dẫn:
 - Đơn giản hóa các biểu thức số học và lôgic.
 - Tính cẩn thận từng chu kì lồng nhau để xác định liệu các câu lệnh hay biểu thức có thể được chuyển ra ngoài hay không

Lập trình - hiệu quả

- Khi có thể, hãy tránh dùng mảng nhiều chiều
- Khi có thể hãy tránh việc dùng con trỏ và danh sách phức tạp
- Dùng các phép toán số học “nhanh”
- Không trộn lẫn các kiểu dữ liệu, cho dù ngôn ngữ có cho phép điều đó

Lập trình - hiệu quả

- Hiệu quả bộ nhớ
 - Hạn chế bộ nhớ trong phát triển phần mềm nhưng là mối quan tâm rất thực tế, mặc dù bộ nhớ giá thấp, mật độ cao vẫn đang tiến hóa nhanh chóng.
 - Nếu yêu cầu hệ thống cần tới bộ nhớ tối thiểu (như sản phẩm giá thấp, khối lượng lớn) thì trình biên dịch ngôn ngữ cấp cao phải được xem xét cẩn thận với tính năng nén bộ nhớ.

Lập trình - hiệu quả

- Hiệu quả vào/ra
 - Số các yêu cầu vào/ra nên giữ mức tối thiểu
 - Mọi việc vào/ra nên qua bộ đệm để làm giảm phí tổn liên lạc.
 - Với bộ nhớ phụ (như đĩa) nên lựa chọn và dùng phương pháp truy xuất đơn giản nhất chấp nhận được.
 - Việc vào/ra với thiết bị cuối hay máy in nên nhận diện các tính năng của thiết bị có thể cải tiến chất lượng hay tốc độ.

Lập trình - hiệu quả

- Ví dụ:

Tính $1!, 2!, 3! \dots n!$

- C1 (giải thông thường)

Viết hàm Float GiaiThua(int k)

Sử dụng hàm này trong vòng lặp

Lặp $I=1, n$

In GiaiThua(I)

Cuối lặp

Cách này không hiệu quả vì độ phức tạp $O(n^2)$

Lập trình - hiệu quả

- C2 (hiệu quả hơn)

Sử dụng tính chất $k! = (k-1)! * k$. độ phức tạp chỉ còn $O(n)$

GT=1

Lặp $I=1, \dots, n$

GT=GT*I

In GT

Cuối Lặp

Lập trình - hiệu quả

- Ví dụ 2:

Tính $S(n,x)=1+x/1!+x^2/2!+..+x^n/n!$

- Cách thông thường:

Chia nhỏ vấn đề, cài đặt các hàm cho từng công việc

$\forall k=0,1,2,...,n$

tính x^k

tính $k!$

Độ phức tạp là $O(n^2)$

Lập trình - hiệu quả

- Cách hiệu quả hơn:

$$\text{xét } a_k = x^k/k! \Rightarrow a_{k+1} = a_k * x/(k+1)$$

$$a_0 = x^0/0! = 1$$

$$S=1; a=1$$

Lặp I=2..,n

$$a = a * x / (I+1)$$

$$S = S + a$$

Cuối Lặp

độ phức tạp $O(n)$

Lập trình - hiệu quả

- Ví dụ 3

Tính $C_n^0, C_n^1, \dots, C_n^n$ khi biết n

Cách thông thường:

- Viết hàm tính giai thừa k
- Tính tổ hợp $C_n^k = n! / (k! * (n-k)!)$

Độ phức tạp $O(n^2)$

Lập trình - hiệu quả

- Các cải tiến
 - C1 (hiệu quả)
 - Dùng mảng $n+1$ phần tử
 $a[0], a[1], \dots, a[n]$
để tính và lưu $a[i]=i!$
Sử dụng vòng lặp n tính C_n^k
Độ phức tạp $O(\alpha n)$ với α hằng

Lập trình - hiệu quả

- C2 (hiệu quả hơn)

Ta có $C_n^k = n! / (k! * (n-k)!)$

$$C_n^{k+1} = n! / ((k+1)! * (n-k-1)!) = n! / (k! * (k+1) * (n-k+1)!)$$

$$\Rightarrow C_n^{k+1} / C_n^k = (n-k) / (k+1) \Rightarrow C_n^{k+1} = ((n-k) / (k+1)) * C_n^k$$

$$C_n^0 = 1$$

Sử dụng vòng lặp để tính

Độ phức tạp $O(n)$ và không hao bộ nhớ vì không dùng mảng

Tổng kết

- Bước lập trình là một tiến trình chuyển hóa thiết kế chi tiết thành chương trình
- Các đặc trưng của ngôn ngữ lập trình có ảnh hưởng lớn đến chương trình
- Phong cách lập trình quyết định tính dễ hiểu của chương trình gốc
- Lập trình cần hướng tới hiệu quả thực thi.